3. Multimodal Visual Understanding + Depth Camera Distance Q&A

Introduction

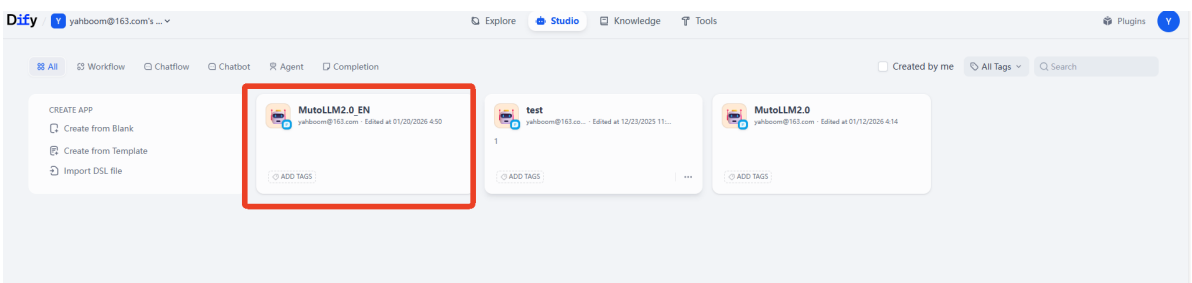
In this tutorial, we will learn how to give instructions by chatting with Muto in the Dify interface.

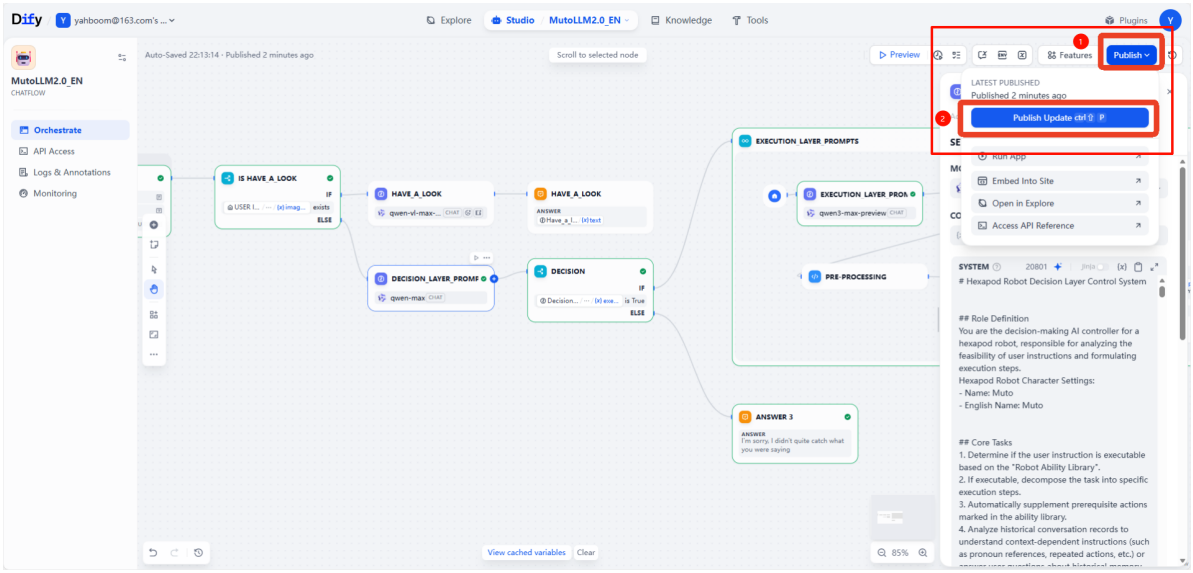
Steps

Prerequisites

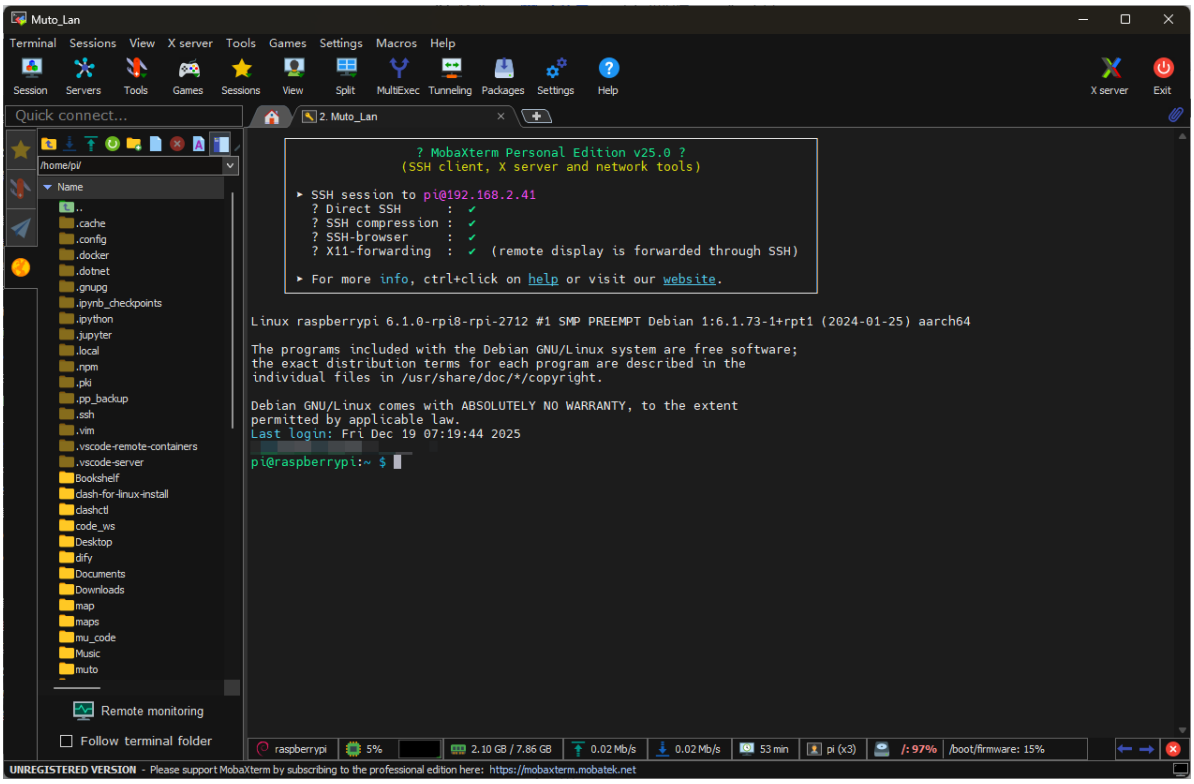
For this course, you need to start the Dify terminal in advance and have all the configuration items set up according to the previous chapters.

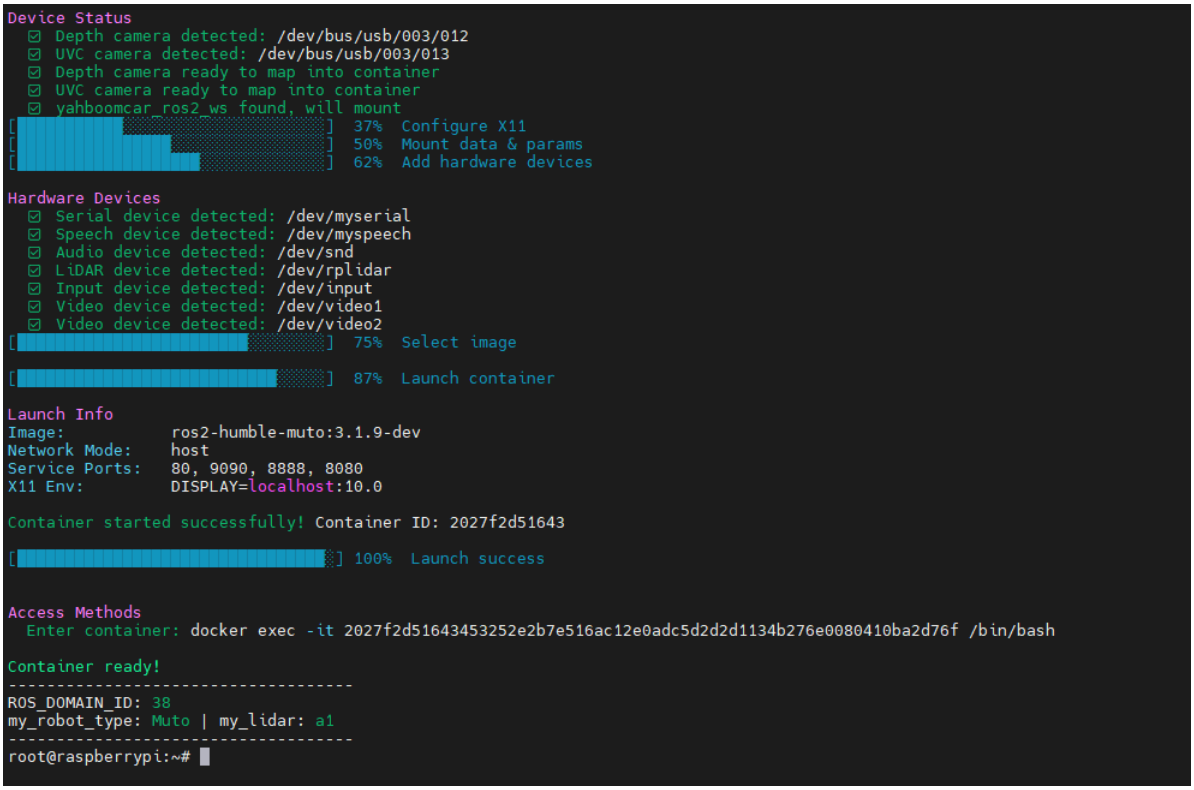
```
#!/usr/bin/env bash

# ==========================================
# User Configuration Area
# ==========================================

# 1. Voice Module Configuration
# ============================

# 1.1 Provider & Keys
export ALIBABA_API_KEY="sk-[redacted]20"

export FORCE_VOICE_PROVIDER="xunfei"

export VOICE_LANGUAGE="en"

# 2. Dify Configuration
# Auto-detect IP from wlan0
export DIFY_HOST=$(ip addr show wlan0 2>/dev/null | grep "inet " | awk '{print $2}' | cut -d/ -f1 | head -n 1)
# Fallback if wlan0 has no IP
if [ -z "$DIFY_HOST" ]; then
    export DIFY_HOST="127.0.0.1"
fi
```





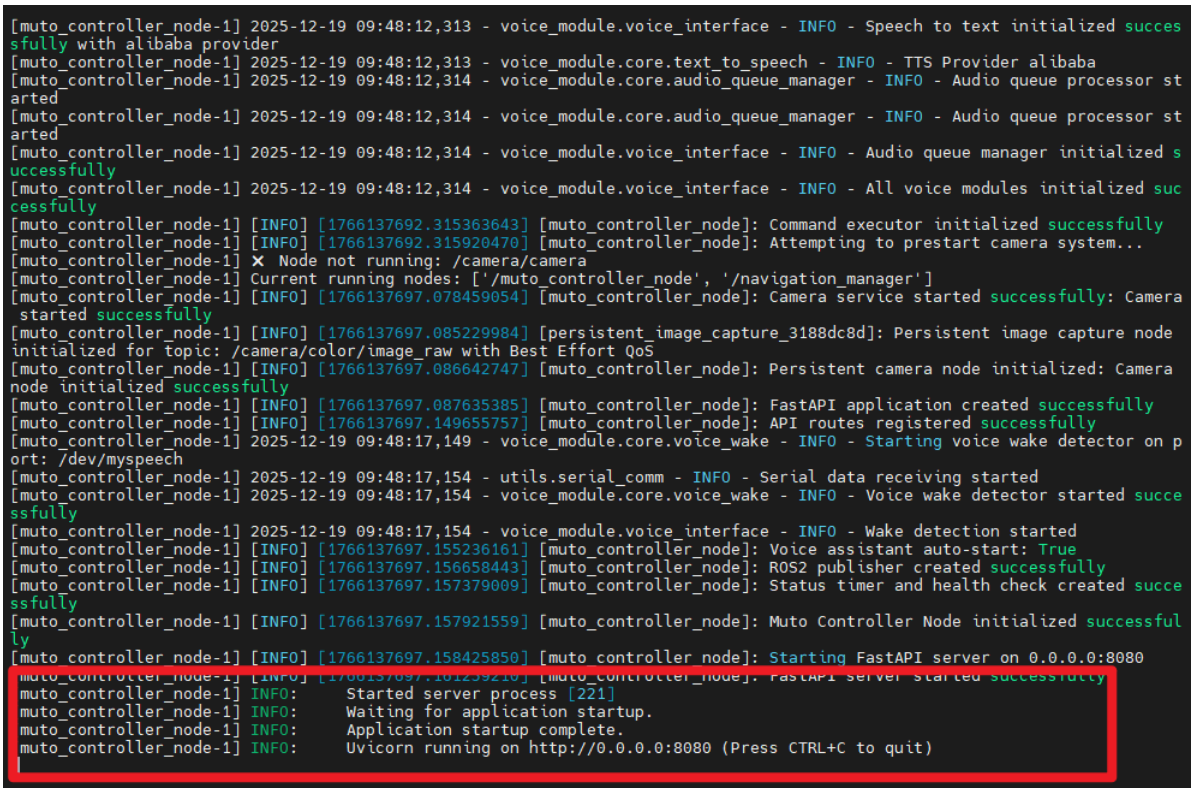
This course requires MutoRS to be powered on. Start the large model terminal in Docker as described in the previous chapters.





In this tutorial, we use the voice version of the large model. **Do not** include any parameters when starting it.

```
./rebuild_and_launch.sh
```



Getting Started

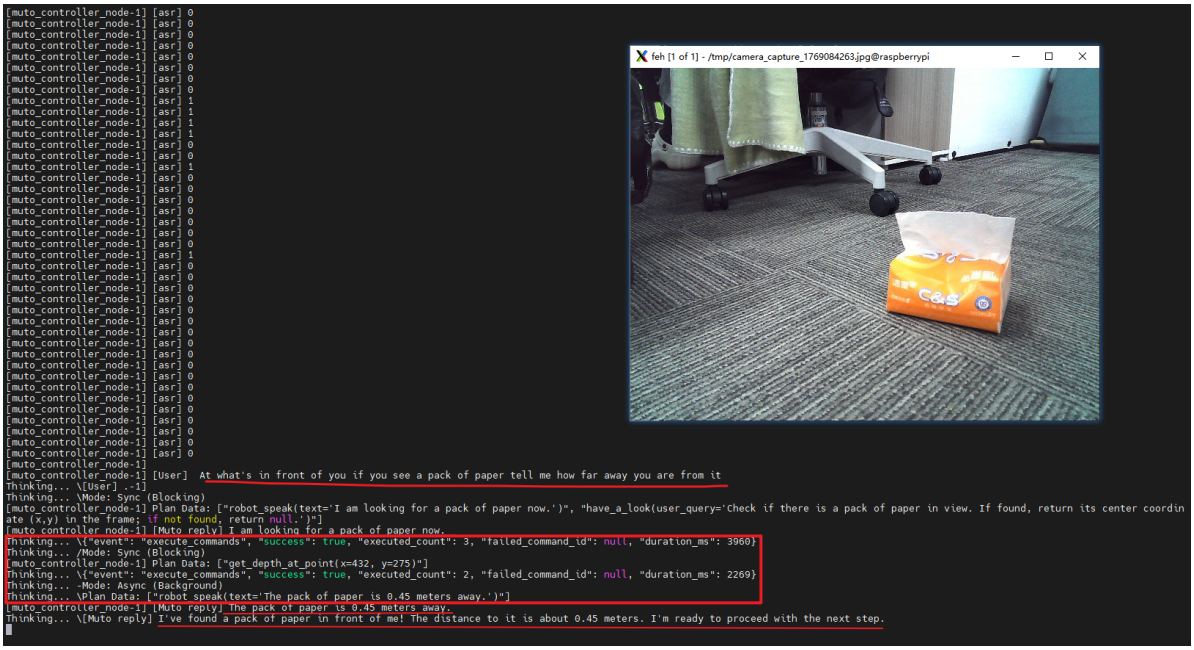
In this tutorial, we will integrate the large model with our depth camera.

Usage Example

The command used in this section is: "Look at what's in front of you. If you see a pack of paper, tell me how far away you are from it."

Input this command via voice.

You can see a visualized image has opened.



Here we have measured the actual distance.

> Note! When using the Astra Pro plus camera on MutoRS for distance measurement, the object needs to be more than 0.8cm away. Otherwise, the obtained depth data may be considered invalid and discarded.

You can change the object in your query based on the actual scene.

Code Briefing

The key function used in this section is:

```
get_depth_at_point(x, y): Get the distance at the specified coordinates (x, y) in the depth camera view (unit: meters). This needs to be used in conjunction with visual recognition results.
```

The function is defined in the path `\packages\muto_hexapod_lib\muto_hexapod_lib\Largemodel\sensor_manager.py`.

The core code is as follows:

```
    def get_depth_at_point(self, x: int, y: int, timeout_s: float = 5.0) ->
tuple:
        """
        Get depth information at specified coordinates (with intelligent node
management)
```

```
        Args:
            x: X coordinate in the image
            y: Y coordinate in the image
            timeout_s: Timeout in seconds

        Returns:
            tuple: (success: bool, message: str, data: dict)
                success: Whether execution was successful
                message: Execution result message
                data: Depth information
        """
        depth_topic = "/camera/depth/image_raw"

        try:
            # Intelligent node management: check initial camera status
            camera_was_running = False
            camera_started_by_us = False

            camera_success, camera_msg, camera_data = self.get_camera_status()
            camera_was_running = camera_success and camera_data.get('status') ==
'running'

            # If camera is not running, start it first
            if not camera_was_running:
                start_success, start_msg, start_data =
self.start_camera(wait_time_s=5.0)
                if not start_success:
                    return False, f"无法启动相机 Failed to start camera:
{start_msg}", {
                        "x": x,
                        "y": y,
                        "camera_start_attempted": True,
                        "camera_start_success": False,
                        "error": "camera_start_failed"
                    }
                camera_started_by_us = True

            # Execute core depth acquisition function
            # Starting to get depth info at point
            # Depth topic

            # Check for necessary dependencies
            try:
                import rclpy
                from rclpy.node import Node
                from sensor_msgs.msg import Image
                from cv_bridge import CvBridge
                import cv2
                import numpy as np
                # All required dependencies available
            except ImportError as import_error:
                # Missing required dependencies
                # Please install: pip install opencv-python
                return False, f"缺少必要依赖 Missing required dependencies:
{str(import_error)}", {"x": x, "y": y, "error": "missing_dependencies"}

            # Create depth image capture node
            import uuid
```

```
            depth_node_id = str(uuid.uuid4())[:8]  # Use first 8 chars of UUID
as unique ID

            class DepthCaptureNode(Node):
                def __init__(self):
                    super().__init__(f'depth_capture_node_{depth_node_id}')
                    self.bridge = CvBridge()
                    self.depth_received = False
                    self.depth_image = None
                    self.subscription = self.create_subscription(
                        Image,
                        depth_topic,
                        self.depth_callback,
                        10
                    )
                    # Subscribed to depth topic

                def depth_callback(self, msg):
                    try:
                        # Convert ROS depth image message to OpenCV image
                        # Depth image is usually 16-bit single channel
                        depth_image = self.bridge.imgmsg_to_cv2(msg,
desired_encoding="passthrough")
                        self.depth_image = depth_image
                        self.depth_received = True
                        # Successfully received depth image
                    except Exception as e:
                        print(f"深度图像转换失败 Depth image conversion failed:
{str(e)}")
                        # Depth image conversion failed

            # Initialize ROS2 (if not already initialized)
            if not rclpy.ok():
                rclpy.init()

            # Create node and wait for depth image
            node = DepthCaptureNode()

            # Waiting for depth image data

            # Wait for depth image data - add error handling and retry mechanism
            start_time = time.time()
            spin_retry_count = 0
            max_spin_retries = 3

            while not node.depth_received and (time.time() - start_time) <
timeout_s:
                try:
                    rclpy.spin_once(node, timeout_sec=0.1)
                    spin_retry_count = 0  # Reset retry count
                except Exception as spin_error:
                    spin_retry_count += 1
                    error_msg = str(spin_error)

                    # Check for "event index too big" error
                    if "event index too big" in error_msg or "index" in
error_msg.lower():
                        if spin_retry_count <= max_spin_retries:
```

```
                            print(f"⚠️ ROS2事件循环错误 (深度获取重试
{spin_retry_count}/{max_spin_retries}): {error_msg}")
                            time.sleep(0.2)  # 短暂延迟后重试
                            continue
                        else:
                            print(f"❌ ROS2事件循环错误超过最大重试次数 (深度获取):
{error_msg}")
                            node.destroy_node()
                            return False, f"ROS2事件循环错误 (深度获取):
{error_msg}", {"x": x, "y": y, "error": "ros2_event_loop_error_depth"}
                    else:
                        # Other types of errors
                        print(f"❌ ROS2 spin_once错误 (深度获取): {error_msg}")
                        node.destroy_node()
                        return False, f"ROS2 spin_once错误 (深度获取):
{error_msg}", {"x": x, "y": y, "error": "ros2_spin_error_depth"}

            if node.depth_received and node.depth_image is not None:
                # Check if coordinates are within image bounds
                height, width = node.depth_image.shape[:2]

                if x < 0 or x >= width or y < 0 or y >= height:
                    # Coordinates out of image bounds
                    # Image size
                    # Requested coordinates
                    node.destroy_node()
                    return False, f"坐标超出图像范围 Coordinates out of bounds:
({x}, {y}), image size: {width}x{height}", {
                        "x": x, "y": y,
                        "image_width": width,
                        "image_height": height,
                        "error": "coordinates_out_of_bounds"
                    }

                # Get depth value at specified coordinates
                depth_value = node.depth_image[y, x]

                # Depth value is usually in millimeters, convert to meters
                if node.depth_image.dtype == np.uint16:
                    # 16-bit depth image, usually in mm
                    depth_meters = float(depth_value) / 1000.0
                    depth_mm = float(depth_value)
                elif node.depth_image.dtype == np.float32:
                    # 32-bit float depth image, usually in meters
                    depth_meters = float(depth_value)
                    depth_mm = float(depth_value) * 1000.0
                else:
                    # Other formats, assume mm
                    depth_meters = float(depth_value) / 1000.0
                    depth_mm = float(depth_value)

                # Check if depth value is valid (0 usually means invalid)
                is_valid = bool(depth_value > 0)  # Convert to native Python
bool

                # Depth information retrieved successfully
                # Coordinates
                # Raw depth value
```

```
                # Depth (meters)
                # Depth (millimeters)
                # Data type
                # Valid

                # Get depth info for surrounding area (3x3 region)
                surrounding_depths = []
                for dy in range(-1, 2):
                    for dx in range(-1, 2):
                        nx, ny = x + dx, y + dy
                        if 0 <= nx < width and 0 <= ny < height:
                            surrounding_depth = node.depth_image[ny, nx]
                            if node.depth_image.dtype == np.uint16:
                                surrounding_depth_m = float(surrounding_depth) /
1000.0
                            elif node.depth_image.dtype == np.float32:
                                surrounding_depth_m = float(surrounding_depth)
                            else:
                                surrounding_depth_m = float(surrounding_depth) /
1000.0
                            surrounding_depths.append({
                                "x": nx, "y": ny,
                                "depth_raw": float(surrounding_depth),
                                "depth_meters": surrounding_depth_m,
                                "valid": bool(surrounding_depth > 0)  # Convert
to native Python bool
                            })

                # Clean up node
                node.destroy_node()

                # Intelligent node management: if we started the camera, close it
after execution
                if camera_started_by_us:
                    try:
                        self.stop_camera(wait_time_s=1.0)
                    except Exception as stop_error:
                        print(f"⚠ 自动关闭相机警告: {stop_error}")

                return True, "深度信息获取成功 Depth information retrieved
successfully", {
                    "x": x,
                    "y": y,
                    "depth_raw": float(depth_value),
                    "depth_meters": depth_meters,
                    "depth_millimeters": depth_mm,
                    "is_valid": is_valid,
                    "image_size": {"width": width, "height": height},
                    "data_type": str(node.depth_image.dtype),
                    "surrounding_area": surrounding_depths,
                    "method": "cv_bridge_depth",
                    "camera_auto_managed": camera_started_by_us
                }
            else:
                # Depth image capture timeout
                # Please check if depth camera is publishing data normally
                node.destroy_node()
```

```
                # Intelligent node management: if we started the camera, close it
after execution
                if camera_started_by_us:
                    try:
                        self.stop_camera(wait_time_s=1.0)
                    except Exception as stop_error:
                        print(f"⚠ 自动关闭相机警告: {stop_error}")

                return False, f"深度图像获取超时 Depth image capture timeout
({timeout_s}s)", {
                    "x": x, "y": y,
                    "timeout": timeout_s,
                    "suggestion": "check_depth_camera_publishing",
                    "camera_auto_managed": camera_started_by_us
                }

        except Exception as e:
            # Error during depth information retrieval
            # Intelligent node management: if we started the camera, close it
after execution
            if camera_started_by_us:
                try:
                    self.stop_camera(wait_time_s=1.0)
                except Exception as stop_error:
                    print(f"⚠ 自动关闭相机警告: {stop_error}")

            return False, f"深度信息获取过程中发生错误 Error during depth information
retrieval: {str(e)}", {
                "x": x, "y": y,
                "error": str(e),
                "camera_auto_managed": camera_started_by_us
            }
```